

Система контроля версий Git

1 Введение в системы контроля версий

Представьте себе команду разработчиков, которая одновременно работает над большим проектом: одни исправляют баги, другие добавляют новые функции, третьи обновляют документацию. Без надёжной системы учёта изменений быстро возникает хаос: чьи правки самые свежие, как объединить правки разных участников, и как вернуться к более ранней, «рабочей» версии кода, если что-то пошло не так? Именно для решения этих задач и существуют **системы контроля версий (Version Control Systems, VCS)**.

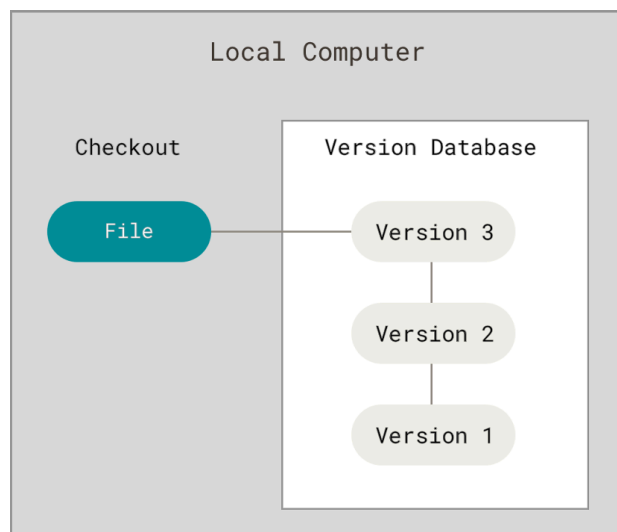
Системы контроля версий позволяют:

- сохранять историю каждого изменения в проекте;
- работать над одним и тем же кодом нескольким людям параллельно, минимизируя риски потери работы и конфликтов;
- быстро откатиться к любой предыдущей версии проекта, если новая реализация оказалась неправильной или сломала что-то важное;
- анализировать, кто и когда внёс то или иное изменение.

Типы систем контроля версий:

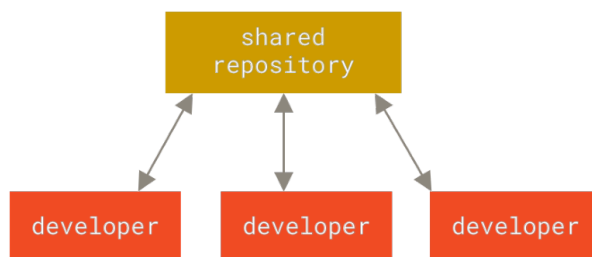
1. Локальные

Всё хранится на компьютере разработчика — удобно, но чревато проблемами при совместной работе, нужно вручную копировать файлы коллегам.



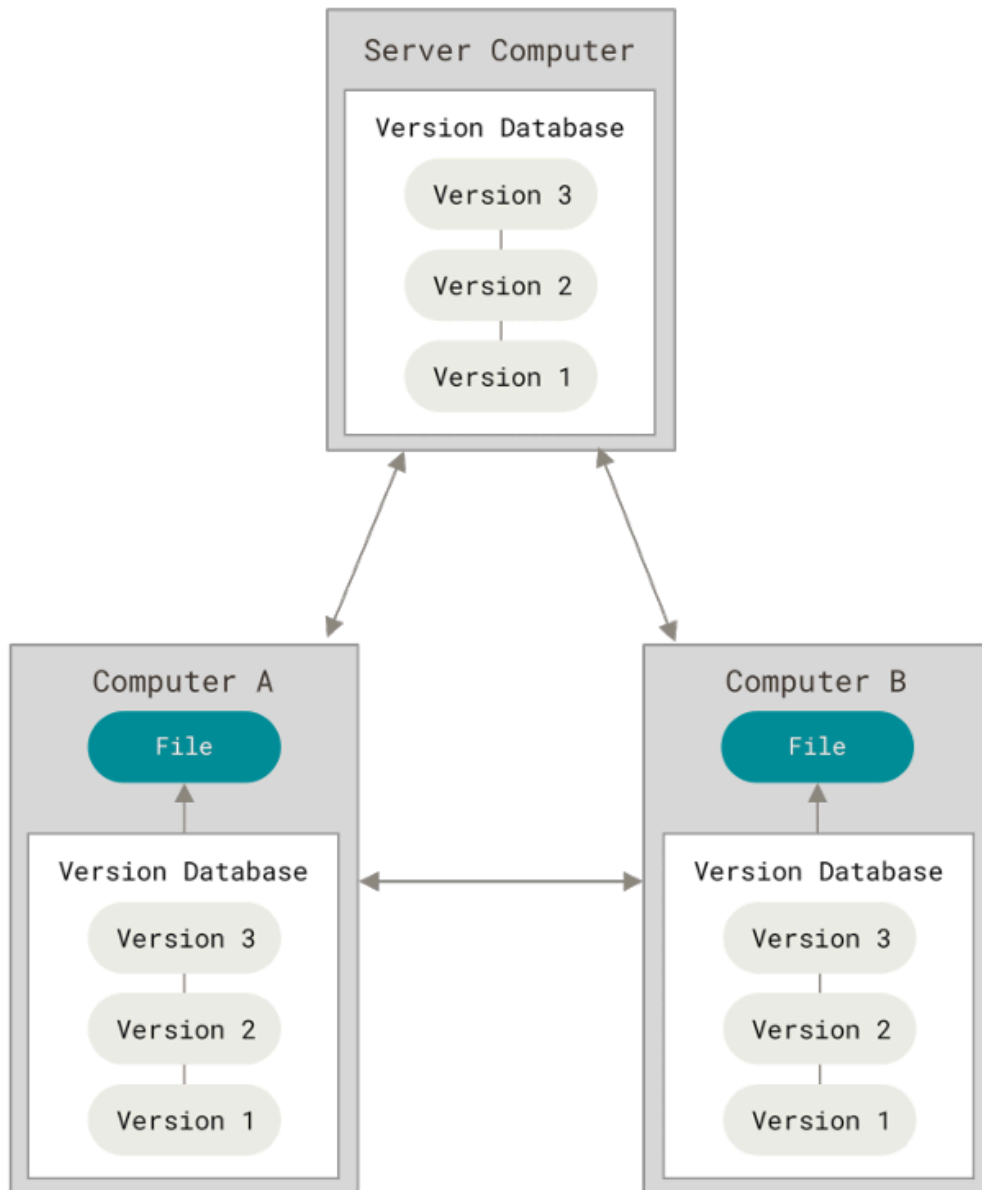
2. Централизованные

Есть единый сервер-репозиторий, к которому все подключаются, история изменений хранится централизованно. Главный плюс — не возникает конфликтов версий, минус — при недоступности сервера вся работа останавливается.



3. Распределённые

Каждый разработчик имеет полную копию всего репозитория, включая всю историю. Они могут вести разработку параллельно и независимо друг от друга в режиме офлайн, периодически проводя синхронизацию.



Основные термины:

- **Репозиторий** — хранилище проекта под управлением системы контроля версий. Содержит файлы, историю изменений и служебную информацию.
- **Коммит (commit)** — зафиксированное изменение в истории проекта. Каждый коммит содержит снимок состояния файлов и сообщение, описывающее внесённые правки.
- **Ветка (branch)** — независимая линия разработки.
- **Слияние (merge)** — объединение изменений из одной ветки в другую.

Почему именно Git?

Git разработан в 2005 году Линусом Торвальдсом — создателем ядра Linux.

Первоначально инструмент создавался для управления исходниками ядра Linux, где ежедневно вносятся тысячи изменений сотнями участников по всему миру. **Git является распределенной системой контроля версий.** На данный момент умение пользоваться Git является необходимым навыком для любого программиста.

Ключевые преимущества Git:

- **Скорость и эффективность.** Операции коммита, ветвления и слияния работают мгновенно, даже в очень больших по размеру репозиториях.
- **Гибкая модель ветвления.** Ветки в Git «лёгкие» и дешёвые по ресурсам, что стимулирует активное использование веток для каждой новой фичи или исправления.
- **Полная локальная копия.** Каждый разработчик имеет свой «клон» репозитория со всей историей, что защищает проект от проблем с центральным сервером.
- **Широкая экосистема.** Над Git построены сервисы (GitHub, GitLab, Bitbucket), графические клиенты, плагины для IDE.
- **Прозрачная история.** Мощные инструменты визуализации, анализа и поиска изменений позволяют легко понимать, кто и зачем внёс те или иные правки.



2 Основы Git

2.1 Установка Git

Скачать Git можно с официального сайта [Git - Downloads](https://git-scm.com/downloads). После установки нужно будет провести первоначальную настройку. Первое, что вам следует сделать после установки Git — указать ваше имя и адрес электронной почты. Это важно, потому что каждый коммит в Git содержит эту информацию, и она включена в коммиты, передаваемые вами, и не может быть далее изменена. Команды нужно вводить в командной строке.

```
git config --global user.name "Имя Фамилия"  
git config --global user.email myemail@example.com
```

2.2 Создание репозитория

Репозиторий Git можно создать одним из двух способов:

1. Вы можете взять локальный каталог (папку), и превратить его в репозиторий Git
2. Вы можете клонировать существующий репозиторий Git из любого места.

Для создания репозитория необходимо открыть командную строку и перейти в каталог, в котором вы собираетесь его создать. В Windows это можно сделать с помощью команды `cd`.

```
cd C:/Users/user/my_project
```

Затем вводите команду для инициализации репозитория:

```
git init
```

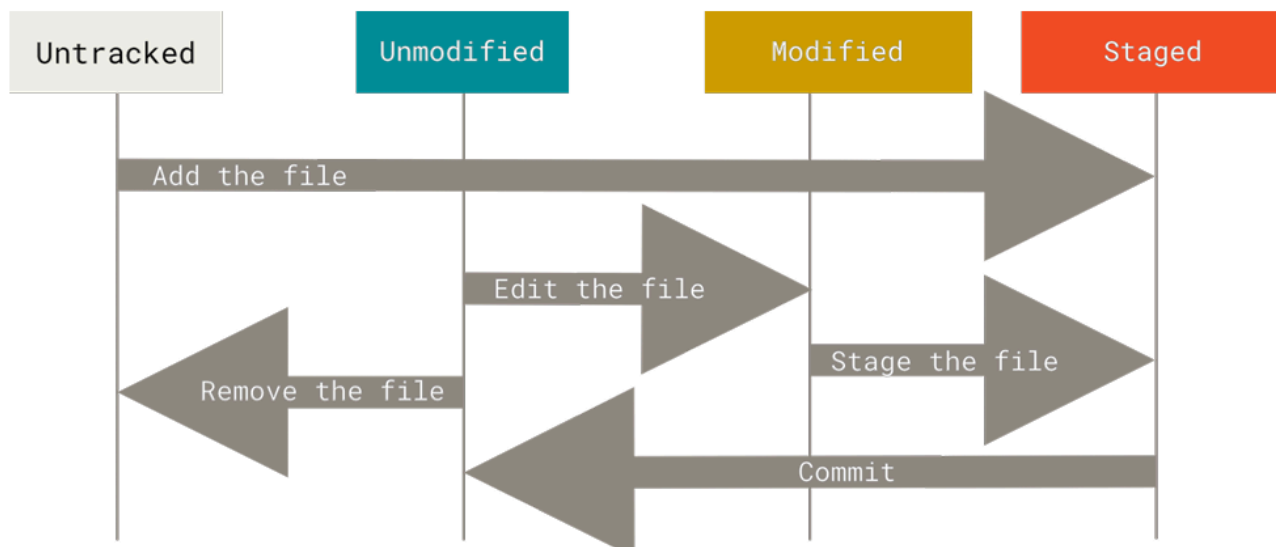
Эта команда создаёт в текущем каталоге новый подкаталог с именем `.git`, содержащий все необходимые файлы репозитория — структуру Git репозитория.

2.3 Запись изменений в репозиторий

Каждый файл в рабочем каталоге может находиться в одном из двух состояний: под версионным контролем (отслеживаемые) и нет (неотслеживаемые).

Отслеживаемые файлы — это те файлы, которые были в последнем снимке состояния проекта, они могут быть **неизменёнными (unmodified)**, **изменёнными (modified)** или **подготовленными к коммиту (staged)**. Если кратко, то отслеживаемые файлы — это те файлы, о которых знает Git.

Неотслеживаемые файлы — это всё остальное, любые файлы в вашем рабочем каталоге, которые не входили в ваш последний снимок состояния и не подготовлены к коммиту. Когда вы впервые создаете репозиторий все файлы будут неотслеживаемыми.



Для того, чтобы понять, какой статус у каждого файла в репозитории нужно использовать команду:

```
git status
```

Для того, чтобы добавить неотслеживаемые файлы в индекс (staged). Нужно использовать команду:

```
git add <file1> <file2> ...
```

Либо

```
git add .
```

Чтобы добавить сразу все неотслеживаемые файлы.

После этой команды можно будет зафиксировать изменения с помощью коммита:

```
git commit -m "Короткое, ёмкое сообщение о сути изменений"
```

Вкратце процесс создания коммита выглядит следующим образом:

1. Вносим изменения в каталоге (добавляем, удаляем, изменяем файлы).
2. Добавляем в индекс `git add`
3. Фиксируем с помощью коммита `git commit`

Для отмены последнего коммита нужно использовать команду:

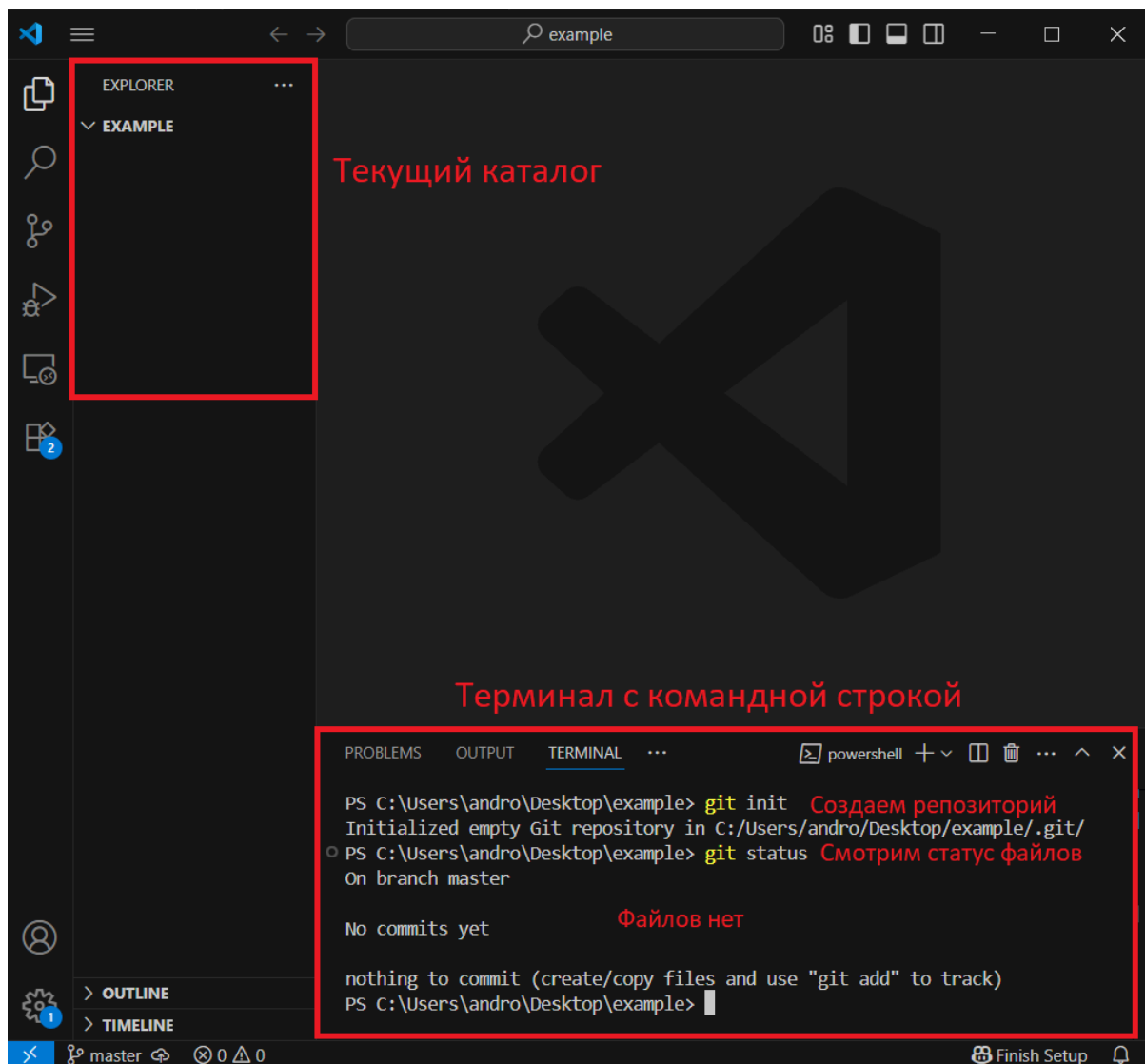
```
git commit --amend
```

К другим полезным командам можно отнести `git diff`, который позволяет увидеть изменения в файлах относительно последнего коммита и `git log` который позволяет увидеть историю коммитов.

Для указания файлов и папок, которые Git не должен отслеживать и включать в коммиты используется файл `.gitignore`. Это особенно полезно для исключения временных файлов, логов, конфигураций среды, скомпилированных бинарников и других данных, не относящихся к исходному коду проекта. Git читает `.gitignore` построчно: каждая строка описывает шаблон имени (например, `*.log` или `node_modules/`). Эти файлы остаются в вашей рабочей директории, но Git будет их игнорировать при добавлении (`git add`) и не включать в коммиты, если они не были добавлены ранее.

2.4 Практический пример

Пример работы с Git с помощью VS Code



EXAMPLE

- text1.txt U
- text2.txt U
- text3.txt U

Добавили несколько файлов

text3.txt

1

PROBLEMS OUTPUT TERMINAL

PS C:\Users\andro\Desktop\example> git status Смотрим статус

On branch master

No commits yet

Untracked files:

(use "git add <file>..." to include in what will be committed)

text1.txt

text2.txt Есть 3 неотслеживаемых файла

text3.txt

nothing added to commit but untracked files present (use "git add" to track)

PS C:\Users\andro\Desktop\example>

master* 0 0

Ln 1, Col 2 Spaces: 4 UTF-8 CRLF {} Plain Text Finish Setup Prettier

```
text1.txt
text2.txt
text3.txt

● nothing added to commit but untracked files present (use "git add" to track)
● PS C:\Users\andro\Desktop\example> git add . Добавили все файлы в индекс
PS C:\Users\andro\Desktop\example> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   text1.txt
    new file:   text2.txt
    new file:   text3.txt
    Теперь они все отслеживаются

● PS C:\Users\andro\Desktop\example> git commit -m "first commit" Сделали коммит
[master (root-commit) 9879c5a] first commit
 3 files changed, 1 insertion(+)
 create mode 100644 text1.txt
 create mode 100644 text2.txt
 create mode 100644 text3.txt

● PS C:\Users\andro\Desktop\example> git status
On branch master
nothing to commit, working tree clean
    Теперь файлы стали unmodified
○ PS C:\Users\andro\Desktop\example>
```

```
PS C:\Users\andro\Desktop\example> git diff
diff --git a/text2.txt b/text2.txt
index f04fd8d..f541c22 100644
--- a/text2.txt
+++ b/text2.txt
@@ -1,1 @@
-qwertyuiop
\ No newline at end of file
+qwertyuiopfgfgfgfgff
\ No newline at end of file
```

```
PS C:\Users\andro\Desktop\example> git log
commit bdfa440e9a11ca65a24a65314a5f2b74054d580e (HEAD -> master) Хэш-код коммита
Author: OrlovAndrei <androrvit@gmail.com> Автор
Date: Sat Jun 28 18:29:00 2025 +0300 Время

    second commit    Описание

commit 9879c5a24feb81095c4a9c675b159403337af484
Author: OrlovAndrei <androrvit@gmail.com>
Date: Sat Jun 28 18:16:36 2025 +0300

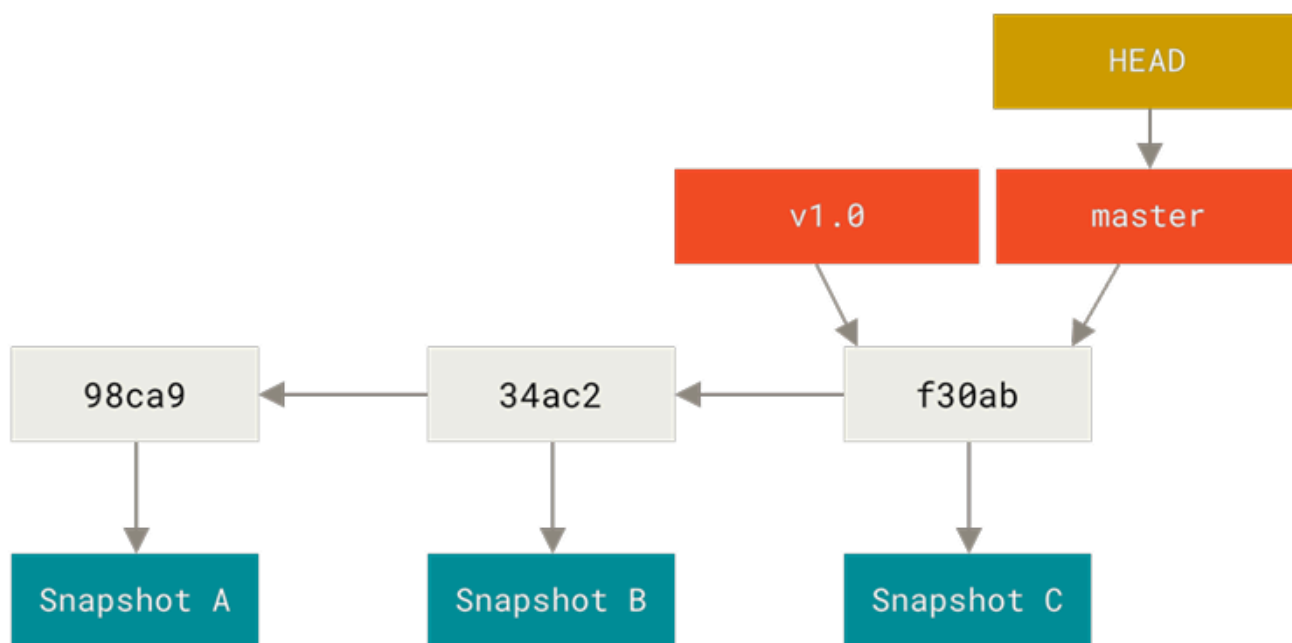
    first commit
```

3 Ветки

Ветки позволяют создавать изолированные копии основной линии разработки, где вы продолжаете работу независимо от неё. Для точного понимания механизма ветвлений, необходимо вернуться назад и изучить то, как Git хранит данные.

Когда вы делаете коммит, Git сохраняет его в виде объекта, у которого есть указатель на коммит или коммиты непосредственно предшествующие данному. Ветка в Git — это простой перемещаемый указатель на один из таких коммитов. По умолчанию, имя основной ветки в Git — `master`. Как только вы начнёте создавать коммиты, ветка `master` будет всегда указывать на последний коммит. Каждый раз при создании коммита указатель ветки `master` будет передвигаться на следующий коммит автоматически.

Рассмотрим пример, у нас есть три последовательных коммита, которые ссылаются на предыдущий. И две ветки `v1.0` и `masters`, которые указывают на один и тот же последний коммит (ветки одинаковые). Так же есть указатель `HEAD`, который указывает на ту ветку, на которой мы сейчас находимся.



3.1 Создание ветки

Для создания новой ветки нужно ввести команду:

```
git branch <название ветки>
```

Это создаст новый указатель, на тот коммит, на котором вы находитесь сейчас.



Но при этом указатель `HEAD` не перенесется автоматически и вы останетесь на той же ветке.

Для того, чтобы переключиться на новую ветку, нужно ввести команду:

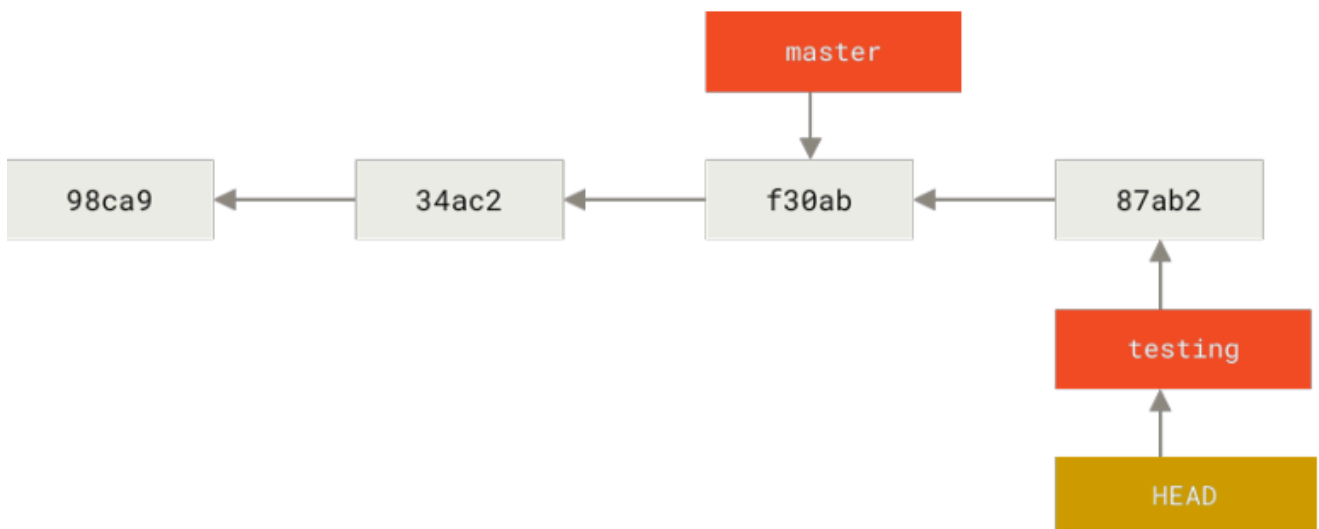
```
git checkout <название ветки>
```

Так же можно написать такую команду для того, чтобы сразу создать и переключиться на новую ветку:

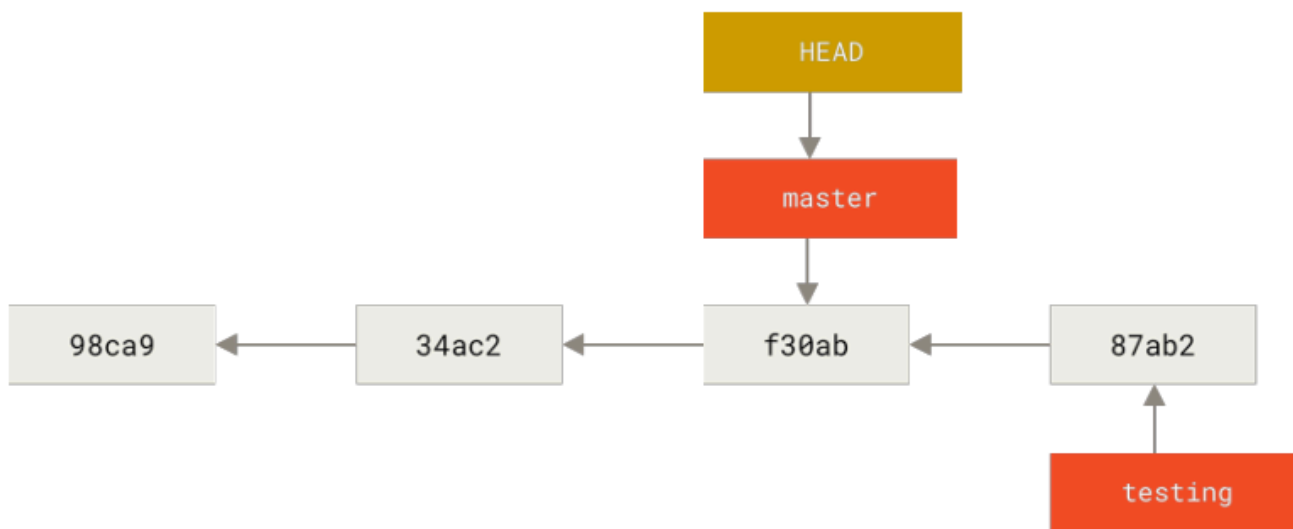
```
git checkout -b <название ветки>
```



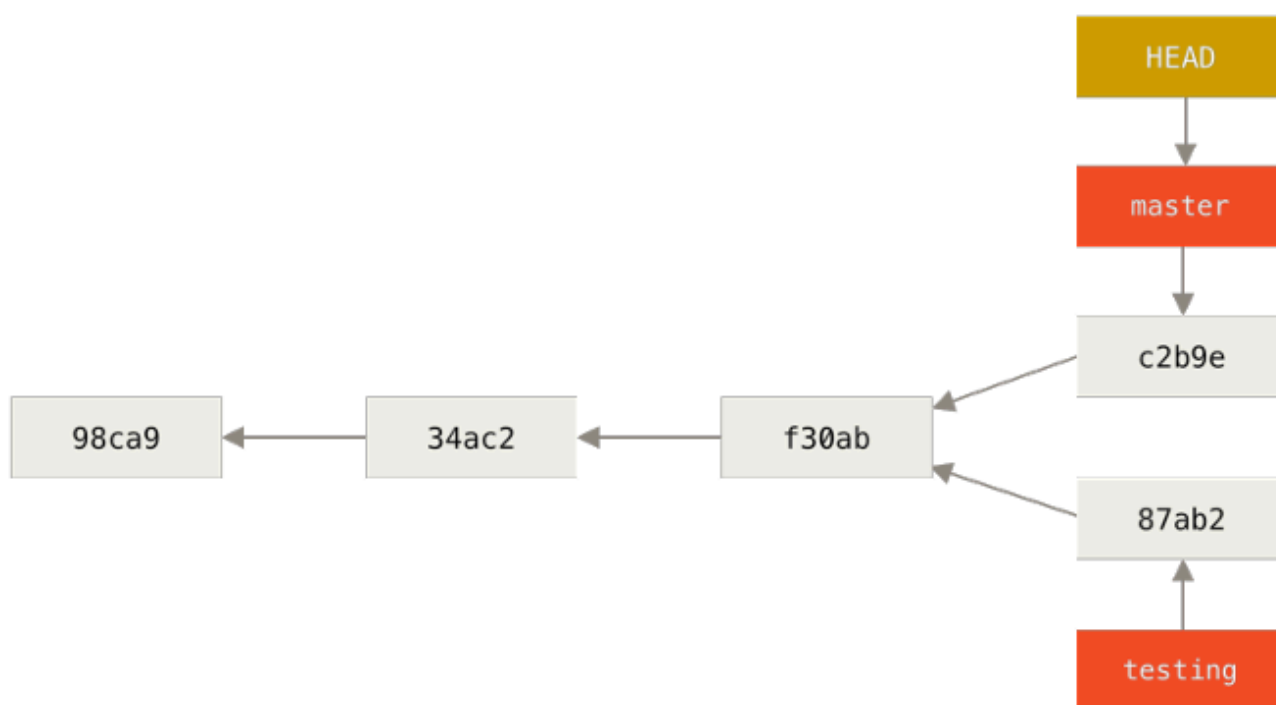
На данный момент ветки идентичные. Но теперь, если находясь в новой ветке, мы добавим коммит, то на него перенесется указатель новой ветки, а указатель `master` останется на месте. Ветки разойдутся, но история коммитов останется линейной.



Теперь, если мы обратно вернемся на ветку `master`.



И сделаем в ней изменения и коммит, то произойдет разветвление истории.



Для того чтобы удалить ветку, нужно использовать команду:

```
git branch -d <название ветки>
```

Для визуализации всех веток и точек слияния, используйте команду:

```
git log --oneline --graph --all
```

Если нужно изменить название ветки, то можно воспользоваться командой:

```
git branch --move <старое название> <новое название>
```

3.2 Слияние веток

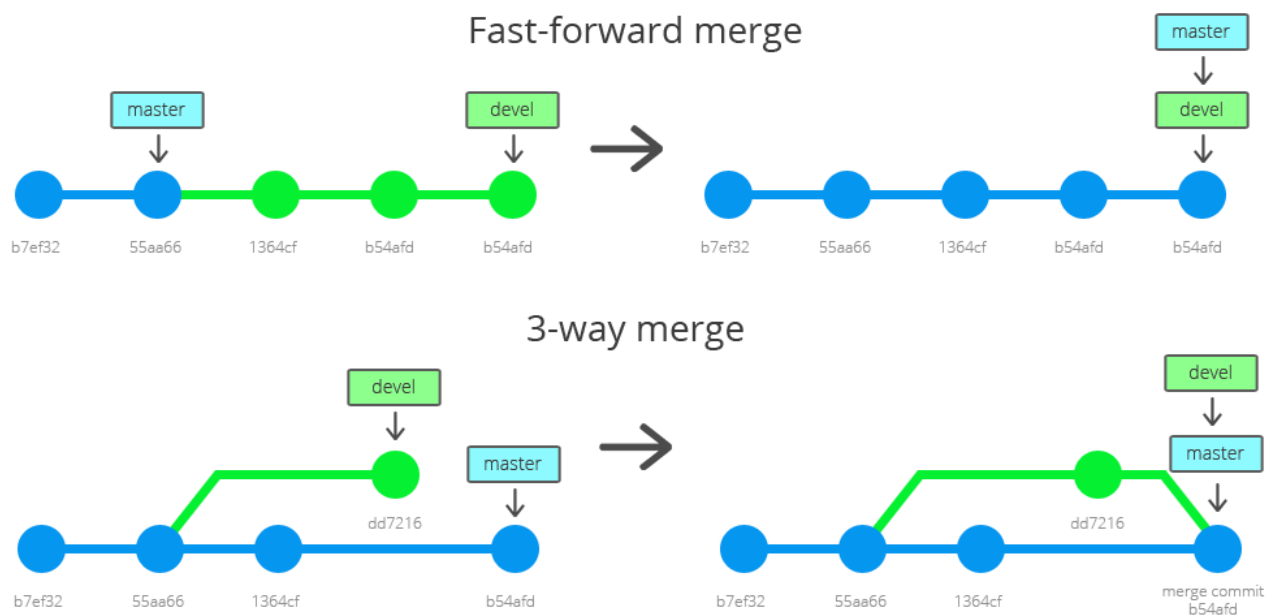
Слияния веток нужны для того, чтобы перенести изменения из одной ветки в другую. Для этого используется команда:

```
git merge <название ветки>
```

Вы должны находиться в той ветке, в которую вам нужно перенести изменения, а в команде указать название ветки, из которой вы хотите эти изменения перенести.

Слияние будет происходить по-разному, в зависимости от того, как расположены ветки относительно друг друга.

- Если указатели находятся на одной линии, то указатель одной ветки просто переносится на место указателя другой, такое слияние называется **fast-forward**.
- Если указатели находятся на разных ветках, то создается новый коммит слияния, который создается на основе последних коммитов из двух веток. Это называется **трехсторонним слиянием**.



При трехстороннем слиянии могут возникнуть **конфликты слияния**. Они возникают тогда, когда один и тот же файл в разных ветках был изменен по-разному и Git не может их автоматически объединить. Git помечает конфликтные участки в файлах специальными маркерами (<<<<<< , ===== , >>>>>>), например:

```
<<<<<< master
изменения которые были внесены в ветке master
=====
изменения которые были внесены в ветке hotfix
>>>>>> hotfix
```

Для того, чтобы разрешить конфликт, нужно

- Открыть файл
- Найти маркеры
- Вручную выбрать или объединить нужные части кода
- Добавить в индекс и закоммитить

3.3 Практический пример

```
text2.txt
1  qwertyuiopfgfgfgfgffdsfsdfsdfsd
```

PROBLEMS OUTPUT TERMINAL ... powershell + - ▢ ▢ ... ^ X

- PS C:\Users\andro\Desktop\example> git checkout -b testing
Switched to a new branch 'testing' Создали новую ветку и переключились на нее
- PS C:\Users\andro\Desktop\example> git commit -am "test change" Добавили в нее коммит
[testing 0eec438] test change
1 file changed, 1 insertion(+), 1 deletion(-)
- PS C:\Users\andro\Desktop\example> git log --oneline --graph --all
* 0eec438 (HEAD -> testing) test change
* bdfa440 (master) second commit Ветки разошлись
* 9879c5a first commit
- PS C:\Users\andro\Desktop\example> git checkout master Вернулись на master
Switched to branch 'master'
- PS C:\Users\andro\Desktop\example> git merge testing Слили с testing
Updating bdfa440..0eec438
Fast-forward
text2.txt | 2 +- Находятся на одной линии, поэтому просто перенесли указатель
1 file changed, 1 insertion(+), 1 deletion(-)
- PS C:\Users\andro\Desktop\example> git log --oneline --graph --all
* 0eec438 (HEAD -> master, testing) test change Теперь обе ветки на одном и том же коммите
* bdfa440 second commit
* 9879c5a first commit
- PS C:\Users\andro\Desktop\example>

> OUTLINE
> TIMELINE

master 0 0

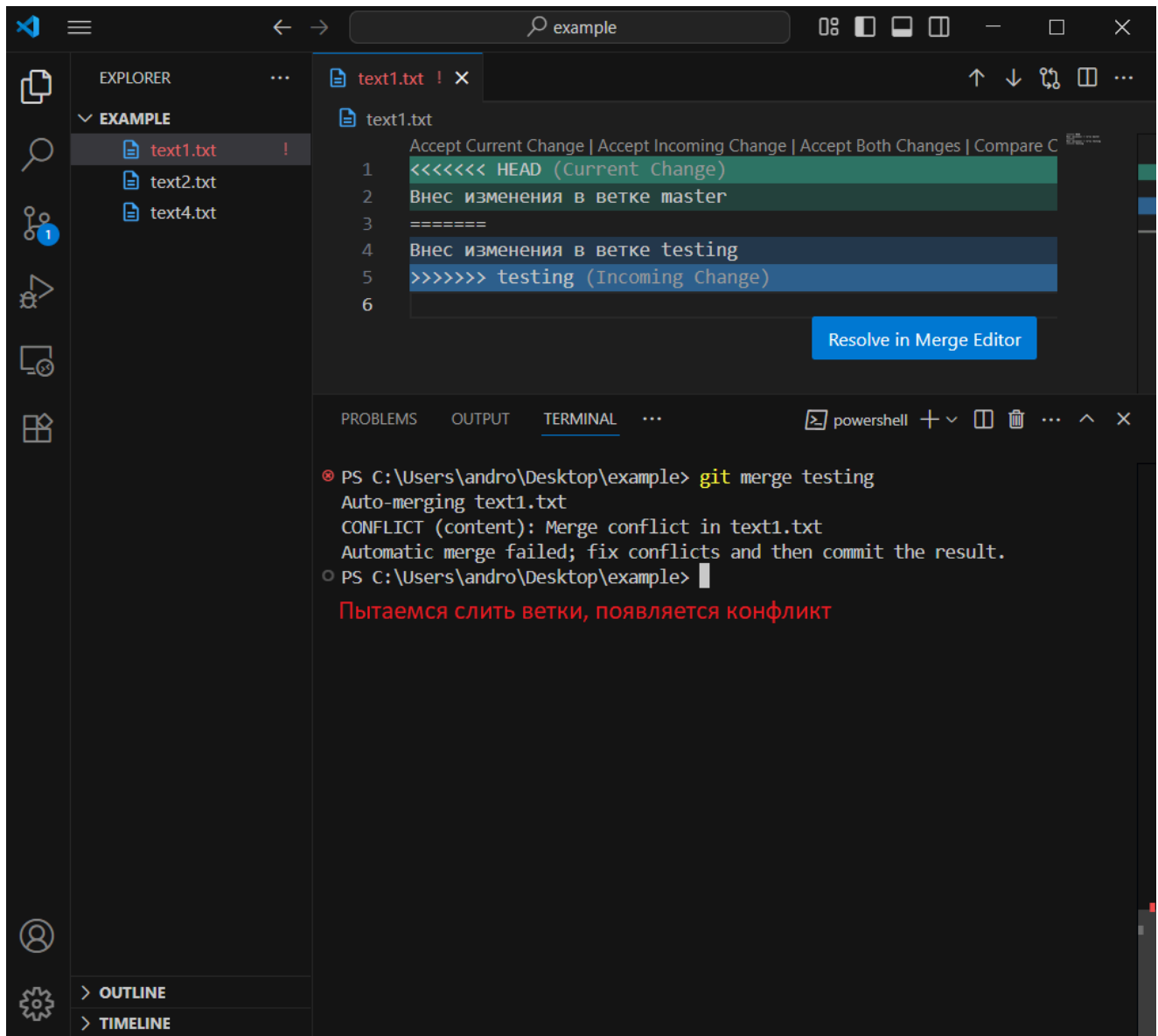
Ln 1, Col 34 Spaces: 4 UTF-8 CRLF {} Plain Text Prettier

Visual Studio Code interface showing a Git workflow. The Explorer sidebar shows a project named 'EXAMPLE' with files 'text1.txt', 'text2.txt', and 'text4.txt'. The main editor shows 'text1.txt' with the content '1 Внес изменения в ветке master'.

The integrated terminal shows the following commands and outputs:

- PS C:\Users\andro\Desktop\example> `git checkout testing`
Switched to branch 'testing'
- PS C:\Users\andro\Desktop\example> `git commit -am "testing change"`
[testing f160337] testing change **Меняем text1 в testing**
1 file changed, 1 insertion(+)
- PS C:\Users\andro\Desktop\example> `git checkout master`
Switched to branch 'master'
- PS C:\Users\andro\Desktop\example> `git commit -am "master change"`
[master c30cf45] master change **Меняем text1 в master**
1 file changed, 1 insertion(+)
- PS C:\Users\andro\Desktop\example> `git log --oneline --graph --all`
* c30cf45 (HEAD -> master) master change
| * f160337 (testing) testing change **Ветки разошлись**
|/
* 0eec438 test change
* bdfa440 second commit
* 9879c5a first commit
- PS C:\Users\andro\Desktop\example>

The status bar at the bottom shows 'master' with 0 errors and 0 warnings. The cursor is at line 1, column 30. The file encoding is UTF-8, line endings are CRLF, and the file is in Plain Text format. The status bar also includes 'Finish Setup' and 'Prettier' icons.



VS Code interface showing a file explorer, a text editor, and a terminal window.

EXPLORER

- EXAMPLE
 - text1.txt
 - text2.txt
 - text4.txt

text1.txt

```
1 Внес изменения в ветке master
2 Внес изменения в ветке testing
3 Убираем все лишние
```

TERMINAL

```
PS C:\Users\andro\Desktop\example> git merge testing
Auto-merging text1.txt
CONFLICT (content): Merge conflict in text1.txt
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\andro\Desktop\example> git commit -am "resolve conflict"
[master 52e093b] resolve conflict
PS C:\Users\andro\Desktop\example> git commit -am "resolve conflict"
[master 6450317] resolve conflict
1 file changed, 3 deletions(-)
PS C:\Users\andro\Desktop\example> git log --oneline --graph --all
* 6450317 (HEAD -> master) resolve conflict
* 52e093b resolve conflict
|\
| * f160337 (testing) testing change
* | c30cf45 master change
|/
* 0eec438 test change
* bdfa440 second commit
* 9879c5a first commit
PS C:\Users\andro\Desktop\example>
```

STATUS BAR

master 0 0 Ln 3, Col 1 Spaces: 4 UTF-8 CRLF {} Plain Text Finish Setup Prettier

4 Работа с удалёнными репозиториями

Мы рассмотрели работу с локальным репозиторием, то есть находящимся у вас на компьютере. Следующим шагом становится взаимодействие с удалёнными хранилищами — серверами, где хранится центральная (или коллективная) копия вашего проекта. Это позволяет нескольким разработчикам совместно работать над кодом.

Самой популярной платформой для размещения репозитория является [GitHub](#). После того, как вы там зарегистрируетесь, у вас будет возможность создать репозиторий. Ему можно дать название и сделать публичным или приватным.

4.1 Подключение к удаленному репозиторию

Чтобы связать локальный проект с удалённым, выполните команду:

```
git remote add origin <URL-репозитория>
```

Где `origin` это псевдоним для удаленного репозитория. Для того, чтобы убедиться, что репозитории связаны, выполните команду:

```
git remote -v
```

Если вы наоборот, хотите создать локальный репозиторий на основе удаленного, то введите команду, которая скопирует его на ваш компьютер:

```
git clone <URL-репозитория>
```

GitHub рекомендует называть основную ветку `main`, а не `master`.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Owner * **Repository name ***

 OrlovAndrei / Test

✔ Test is available.

Great repository names are short and memorable. Need inspiration? How about [redesigned-bassoon](#) ?

Description (optional)

test repo

☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: **None**

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: **None**

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

Основными командами являются:

- **git push** — отправляет ваши локальные коммиты в удалённый репозиторий. При первом пуше в ветку `main` (или `master`) обычно используют: `git push -u origin master` дальнейшем `git push` можно вызывать без параметров, также GitHub попросит вас авторизоваться.
- **git fetch** — загружает новые коммиты из удалённого репозитория, но не сливает их с вашей текущей веткой. Это безопасный способ проверить, что изменилось на сервере: `git fetch origin`
- **git pull** — сочетает `fetch` и `merge` : сразу подтягивает коммиты и объединяет их с вашей веткой: `git pull`

При подключении к удаленному репозиторию, в репозитории появятся **ветки слежения**, которые служат для связи с удаленными ветками. Имена веток слежения имеют вид `<remote>/<branch>` .

The image shows a screenshot of a development environment. The top half is Visual Studio Code, and the bottom half is the GitHub web interface.

Visual Studio Code:

- EXPLORER:** Shows a folder named 'EXAMPLE' containing files 'text1.txt', 'text2.txt', and 'text4.txt'.
- text1.txt:** Contains the following text:

```
1  Внес изменения в ветке master
2  Внес изменения в ветке testing
3
```
- TERMINAL:** Shows a PowerShell session with the following commands and output:

```
PS C:\Users\andro\Desktop\example> git remote add origin https://github.com/OrlovAndrei/Test.git
PS C:\Users\andro\Desktop\example> git remote -v
origin https://github.com/OrlovAndrei/Test.git (fetch)
origin https://github.com/OrlovAndrei/Test.git (push)
PS C:\Users\andro\Desktop\example> git push -u origin master
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (22/22), 1.91 KiB | 980.00 KiB/s, done.
Total 22 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/OrlovAndrei/Test.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
PS C:\Users\andro\Desktop\example> git log --oneline --graph --all
* 6450317 (HEAD -> master, origin/master) resolve conflict
* 52e093b resolve conflict
| \
| * f160337 (testing) testing change
| * | c30cf45 master change
| /
* 0eec438 test change
* bdfa440 second commit
* 9879c5a first commit
PS C:\Users\andro\Desktop\example>
```

GitHub:

- Repository:** OrlovAndrei/Test (Public)
- Branches:** 1 Branch (master)
- Tags:** 0 Tags
- Commits:** 7 Commits
- Files:** text1.txt (resolve conflict, 1 hour ago), text2.txt (test change, 1 hour ago), text4.txt (second commit, yesterday)
- README:** A section titled 'Add a README' with a button 'Add a README'.
- About:** test repo, Activity, 0 stars, 0 watching, 0 forks.
- Releases:** No releases published, Create a new release.
- Packages:** No packages published, Publish your first package.

4.2 README

Помимо кода, каждый проект нуждается в понятной и структурированной документации. Самый распространённый способ оформить описание проекта —

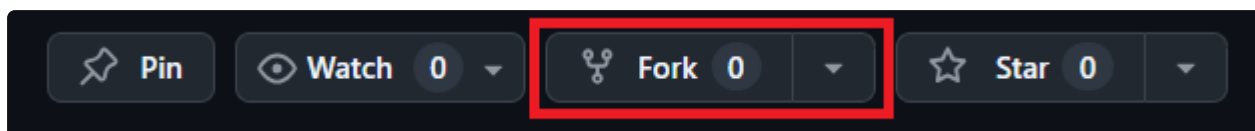
создать файл `README.md` в корне репозитория. Он автоматически отображается на главной странице проекта в GitHub.

Формат `.md` — это **Markdown**, лёгкий язык разметки, который позволяет удобно оформлять текст. Изучить синтаксис Markdown можно [здесь](#).

4.3 Форки и Pull/Merge-запросы

В больших проектах (особенно с открытым исходным кодом) часто используется подход «fork & pull request»:

1. **Fork** — вы создаёте свою копию чужого репозитория на платформе (например, GitHub), чтобы иметь право пушить в неё.
2. **Clone** — клонируете ваш форк локально.
3. **Создаёте ветку** для новой функциональности или исправления:
4. **Делаете коммиты** и пушите ветку в ваш форк:
5. **Открываете Pull Request** (или Merge Request) в исходный репозиторий, где другие участники смогут просмотреть изменения, обсудить их и принять.



Этот подход позволяет защитить главный репозиторий от прямых, возможно неблагонадёжных коммитов и обеспечивает централизованное обсуждение каждого изменения.

Ссылки

1. Скачать Git — [Git - Downloads](#)
 2. Книга по Git, рекомендую прочитать разделы: Введение, Основы Git, Ветвление Git, Распределенный Git, GitHub — [Git](#)
 3. [GitHub](#)
 4. Синтаксис Markdown — [Basic writing and formatting syntax - GitHub Docs](#)
 5. Наглядное приложение для изучения Git — [Learn Git Branching](#)
-

Практическое задание

Что нужно сделать:

1. Создайте новый локальный репозиторий.
2. Создайте два текстовых файла, добавьте их в репозиторий и сделайте первый коммит.
3. Добавьте файл `.gitignore`, в который включите шаблон для игнорирования определённых файлов.
4. Измените один из отслеживаемых файлов и сделайте второй коммит.
5. Создайте новую ветку для изменений.
Переключитесь в неё, внесите изменения в другой файл и сделайте коммит.
6. Вернитесь в основную ветку и выполните слияние с новой веткой.
Слияние должно быть fast-forward (без конфликтов).
7. Сделайте ещё одно изменение в основной ветке и зафиксируйте его коммитом.
8. Перейдите обратно в новую ветку и измените тот же файл, что и в основной ветке.
Выполните коммит, а затем попробуйте слить ветки. Должен произойти конфликт. Разрешите его вручную и сделайте коммит.
9. Просмотрите историю коммитов с отображением ветвлений.
10. Зарегистрируйтесь на GitHub (если ещё не зарегистрированы) и создайте удалённый репозиторий.
Свяжите его с локальным и отправьте в него все коммиты.
11. На GitHub создайте файл `README.md` с кратким описанием проекта.
12. В локальном репозитории получите изменения с GitHub с помощью команды `pull`, чтобы скачать файл `README.md`.

Дополнительно (обязательно):

- **Каждый шаг задания необходимо заскриншотить.**

Скриншоты должны подтверждать выполнение всех действий — содержание файлов, команды в терминале, изменения на GitHub и т.д.

- **Оформите все скриншоты в одном PDF-файле.**

Упорядочьте их по шагам, добавьте подписи (по желанию — краткие комментарии).

- **Добавьте получившийся PDF-отчёт в ваш GitHub-репозиторий.**

Файл можно назвать, например, `report.pdf` и поместить в корень проекта.